

APB Protocol Implementation

Objective

To design and verify an Advanced Peripheral Bus (APB) RAM module using Verilog HDL that supports register-mapped read and write operations following the AMBA APB protocol.

Introduction

The **Advanced Peripheral Bus (APB)** is part of ARM's Advanced Microcontroller Bus Architecture (AMBA). It is designed for communication with low-bandwidth peripherals such as timers, GPIOs, UARTs, and control registers. APB provides a simple, low-power, and non-pipelined interface that requires minimal hardware, making it suitable for peripheral devices where high-speed communication is not required.

The APB protocol uses a two-phase transaction consisting of a **Setup Phase** and an **Access Phase**. During the setup phase, the master provides the address and control signals, while in the access phase the slave completes the read or write operation.

Register Mapped Peripheral

Definition

A **Register Mapped Peripheral** is a hardware device whose internal registers are assigned unique memory addresses. The processor accesses these registers through normal memory read and write operations instead of using separate I/O instructions. This approach simplifies communication between the processor and peripherals.

In this implementation, a RAM array containing **32 memory locations**, each **32 bits wide**, acts as the register-mapped peripheral.

APB Signals:

Signal	Direction	Description
PCLK	Input	System clock
PRESETn	Input	Active-low reset
PSEL	Input	Peripheral select signal
PENABLE	Input	Enables data transfer
PWRITE	Input	Write control (1 = Write, 0 = Read)
PADDR	Input	Address bus
PWDATA	Input	Write data
PRDATA	Output	Read data
PREADY	Output	Indicates transaction completion
PSLVERR	Output	Indicates an invalid transfer

Design Implementation

The APB RAM module was designed using a finite state machine (FSM) with three operating states.

Step 1: Memory Declaration

A memory array of 32 locations is declared to store 32-bit data values.

```
reg [31:0] memory [0:31];
```

Step 2: State Machine Design

The APB slave operates using three states:

- **IDLE:** Waits for the peripheral select signal.
- **SETUP:** Receives the address and control signals.
- **ACCESS:** Performs the read or write operation and completes the transfer.

Step 3: Reset Operation

When **PRESETn** is asserted low:

- Current state is set to **IDLE**.
- All memory locations are cleared.
- Output signals are initialized.

Step 4: Write Operation

During the Access phase:

- If **PWRITE = 1**
- Address is valid
- Data from **PWDATA** is stored into the selected memory location.
- **PREADY** is asserted to indicate successful completion.

Step 5: Read Operation

If **PWRITE = 0**:

- The data stored at the selected address is transferred to **PRDATA**.
- **PREADY** is asserted after successful completion.

Step 6: Error Handling

If the address exceeds the available memory range (0–31):

- **PSLVERR** is asserted.
- **PRDATA** is cleared.
- No memory access is performed.

Code:

Design.v

```
module apb_ram (
    input                pclk,
    input                presetn,
    input                psel,
    input                penable,
    input                pwrite,
    input                [31:0] paddr,
    input                [31:0] pdata,
    output reg [31:0] prdata,
    output reg           pready,
    output reg           pslverr
);

    // Memory Declaration
    reg [31:0] memory [0:31];
    // State Declaration
    localparam IDLE    = 2'b00,
                SETUP  = 2'b01,
                ACCESS = 2'b10;

    reg [1:0] current_state;
    integer i;
    // APB State Machine
    always @(posedge pclk or negedge presetn)
    begin
        if (!presetn)
            begin
                current_state <= IDLE;
                prdata        <= 32'd0;
                pready        <= 1'b0;
                pslverr       <= 1'b0;
                for(i=0;i<32;i=i+1)
                    memory[i] <= 32'd0;
            end
        else
            begin
```

```

    case(current_state)
    // IDLE STATE
    IDLE:
    begin
        pready    <= 1'b0;
        pslverr    <= 1'b0;
        if(psel)
            current_state <= SETUP;
    end
    // SETUP STATE
    SETUP:
    begin
        if(penable)
            current_state <= ACCESS;
    end
    // ACCESS STATE
    ACCESS:
begin
    pready <= 1'b1;
    if(paddr < 32)
    begin
        if(pwrite)
        begin
            memory[paddr] <= pwdata;
        end
        else
        begin
            prdata <= memory[paddr];
        end
        pslverr <= 1'b0;
    end
    else
    begin

```

```

        prdata    <= 32'd0;
        pslverr   <= 1'b1;
    end
    current_state <= IDLE;
end
// DEFAULT
default:
begin
    current_state <= IDLE;
end
endcase
end
end
endmodule

```

Tb.v

```

`timescale 1ns/1ps
module apb_ram_tb;
    reg        pclk;
    reg        presetn;
    reg        psel;
    reg        penable;
    reg        pwrite;
    reg [31:0] paddr;
    reg [31:0] pwrdata;
    wire [31:0] prdata;
    wire        pready;
    wire        pslverr;
    // Instantiate DUT
    apb_ram DUT (
        .pclk(pclk),
        .presetn(presetn),
        .psel(psel),

```

```

        .penable(penable),
        .pwrite(pwrite),
        .paddr(paddr),
        .pwwdata(pwwdata),
        .prdata(prdata),
        .pready(pready),
        .pslverr(pslverr)
    );
    // Clock Generation
    initial
        pclk = 0;
    always #5 pclk = ~pclk;
    // Write Task
    task apb_write;
        input [31:0] address;
        input [31:0] data;
    begin

```

```

        @(posedge pclk);
        psel      = 1;
        penable   = 0;
        pwrite    = 1;
        paddr     = address;
        pwwdata   = data;
        @(posedge pclk);
        penable   = 1;
        @(posedge pclk);
        psel      = 0;
        penable   = 0;
        pwrite    = 0;
    end
    endtask
    // Read Task
    task apb_read;
        input [31:0] address;
    begin

```

```
begin
    @(posedge pclk);
    psel    = 1;
    penable = 0;
    pwrite  = 0;
    paddr   = address;
    @(posedge pclk);
    penable = 1;
    @(posedge pclk);
    $display("Time=%0t  Address=%0d  Data=%h  Ready=%b  Error=%b",
             $time, address, prdata, pready, pslverr);
    psel    = 0;
    penable = 0;
end
endtask
// Test Sequence
initial
```

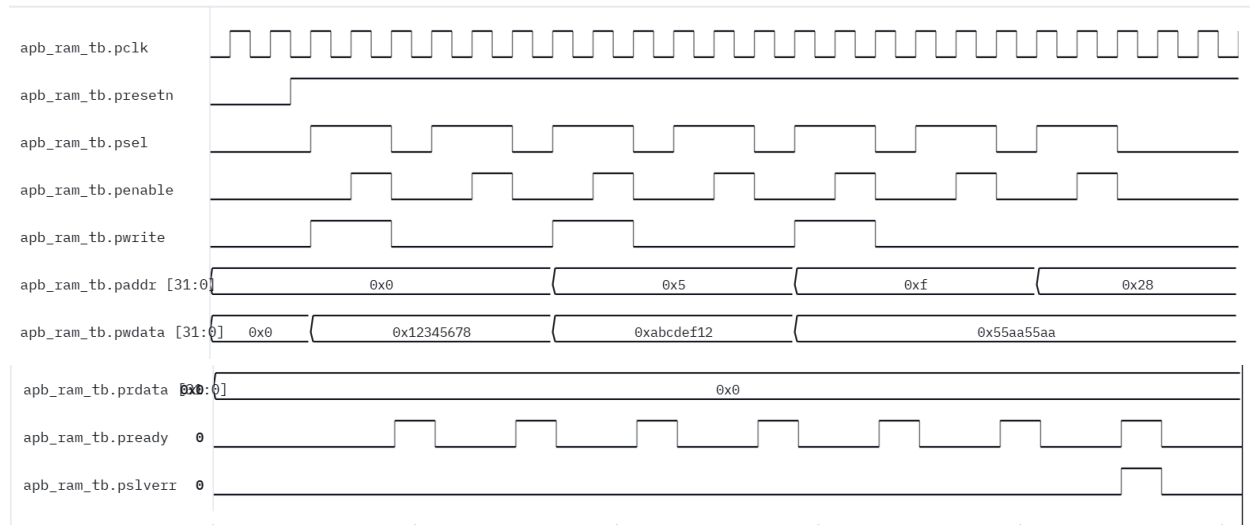
```
begin
    $dumpfile("dump.vcd");
    $dumpvars(0, apb_ram_tb);
    presetn = 0;
    psel     = 0;
    penable  = 0;
    pwrite   = 0;
    paddr    = 0;
    pwdata   = 0;
    // Reset
    #20;
    presetn = 1;
    // Test Case 1
    // Write Register 0
    apb_write(0, 32'h12345678);
    // Test Case 2
    // Read Register 0
```

```
    apb_read(0);  
    // Test Case 3  
    // Write Register 5  
    apb_write(5, 32'hABCDEF12);  
    // Test Case 4  
    // Read Register 5  
    apb_read(5);  
    // Test Case 5  
    // Write Register 15  
    apb_write(15, 32'h55AA55AA);  
    // Test Case 6  
    // Read Register 15  
    apb_read(15);  
  
    // Test Case 7  
    // Invalid Address  
    apb_read(40);  
    #30;  
    $finish;  
  
end  
endmodule
```

Output:

```
VCD info: dumpfile dump.vcd opened for output.  
Time=75000  Address=0  Data=00000000  Ready=0  Error=0  
Time=135000 Address=5  Data=00000000  Ready=0  Error=0  
Time=195000 Address=15 Data=00000000  Ready=0  Error=0  
Time=225000 Address=40 Data=00000000  Ready=0  Error=0  
/tb.v:103: $finish called at 255000 (1ps)  
timings ms: preprocess=14.9 compile=118 simulate=48.1 vcc
```

Waveform:



Conclusion

The APB RAM module was successfully designed and implemented using Verilog HDL based on the AMBA APB protocol. The design supports register-mapped memory with 32 memory locations and correctly performs both read and write operations through the APB interface. A finite state machine was used to control the protocol states, ensuring proper execution of the Setup and Access phases. The developed testbench verified reset functionality, valid read/write transactions, and invalid address handling. Simulation results matched the expected behavior, confirming correct data transfers, proper assertion of the **PREADY** signal upon successful transactions, and generation of the **PSLVERR** signal for invalid accesses. Overall, the implementation demonstrates the fundamental operation of an APB slave peripheral and provides a strong foundation for integrating low-speed peripherals in System-on-Chip (SoC) designs.